9 Sending messages asynchronously

This chapter covers

- Asynchronous messaging
- Sending messages with JMS, RabbitMQ, and Kafka
- Pulling messages from a broker
- Listening for messages

It's 4:55 p.m. on Friday. You're minutes away from starting a much-anticipated vacation. You have just enough time to drive to the airport and catch your flight. But before you pack up and head out, you need to be sure your boss and colleagues know the status of the work you've been doing so that on Monday they can pick up where you left off. Unfortunately, some of your colleagues have already skipped out for the weekend, and your boss is tied up in a meeting. What do you do?

The most practical way to communicate your status and still catch your plane is to send a quick email to your boss and your colleagues, detailing your progress and promising to send a postcard. You don't know where they are or when they'll read the email, but you do know they'll eventually return to their desks and read it. Meanwhile, you're on your way to the airport.

Synchronous communication, which is what we've seen with REST, has its place. But it's not the only style of interapplication communication available to developers. *Asynchronous* messaging is a way of indirectly sending messages from one application to another without waiting for a response. This indirection affords looser coupling and greater scalability between the communicating applications.

In this chapter, we're going to use asynchronous messaging to send orders from the Taco Cloud website to a separate application in the Taco Cloud kitchens where the tacos will be prepared. We'll consider three options that Spring offers for asynchronous messaging: the Java Message Service (JMS), RabbitMQ and Advanced Message Queueing Protocol (AMQP), and Apache Kafka. In addition to the basic sending and receiving of messages, we'll look at Spring's support for message-driven POJOs: a way to receive messages that resembles Enterprise JavaBeans' message-driven beans (MDBs).

9.1 Sending messages with JMS

JMS is a Java standard that defines a common API for working with message brokers. First introduced in 2001, JMS has been the go-to approach for asynchronous messaging in Java for a very long time. Before JMS, each message broker had a proprietary API, making an application's messaging code less portable between brokers. But with JMS, all compliant implementations can be worked with via a common interface in much the same way that JDBC has given relational database operations a common interface.

Spring supports JMS through a template-based abstraction known as `JmsTemplate`. Using `JmsTemplate`, it's easy to send messages across queues and topics from the producer side and to receive those messages on the consumer side. Spring also supports the notion of message-driven POJOs: simple Java objects that react to messages arriving on a queue or topic in an asynchronous fashion.

We're going to explore Spring's JMS support, including `JmsTemplate` and message-driven POJOs. Our focus will be on Spring's support for messaging with JMS, but if you want to know more about JMS, then have a look at *ActiveMQ in Action* by Bruce Snyder, Dejan Bosanac, and Rob Davies (Manning, 2011).

Before you can send and receive messages, you need a message broker that's ready to relay those messages between producers and consumers. Let's kick off our exploration of Spring JMS by setting up a message broker in Spring.

9.1.1 Setting up JMS

Before you can use JMS, you must add a JMS client to your project's build. With Spring Boot, that couldn't be any easier. All you need to do is add a starter dependency to the build. First, though, you must decide whether you're going to use Apache ActiveMQ, or the newer Apache ActiveMQ Artemis broker.

If you're using ActiveMQ, you'll need to add the following dependency to your project's pom.xml file:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-activemq</artifactId>
</dependency>
```

If ActiveMQ Artemis is the choice, the starter dependency should look like this:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-artemis</artifactId>
</dependency>
```

When using the Spring Initializr (or your IDE's frontend for the Initializr), you can also select either of these options as starter dependencies for your project. They are listed as "Spring for Apache ActiveMQ 5" and "Spring for Apache ActiveMQ Artemis," as shown in the screenshot in figure 9.1 from **https://start.spring.io**.

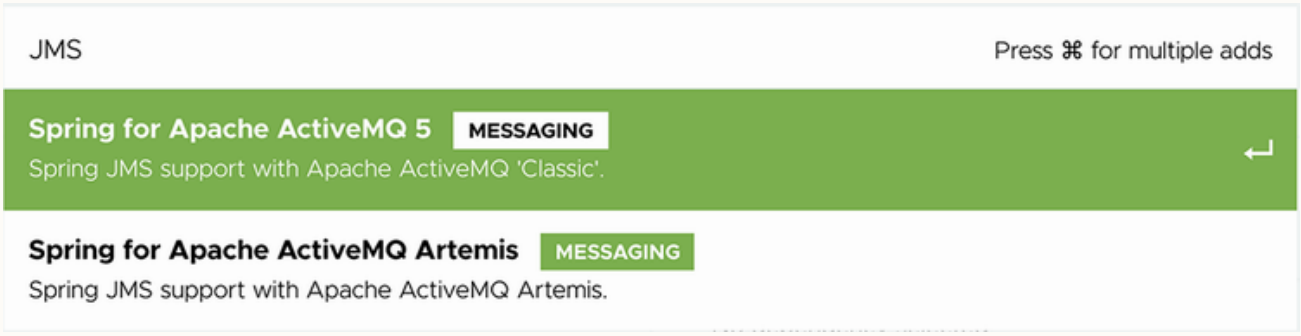


Figure 9.1 ActiveMQ and Artemis choices available in the Spring Initializr

Artemis is a next-generation reimplementation of ActiveMQ, effectively making ActiveMQ a legacy option. Therefore, for Taco Cloud you're going to choose Artemis. But the choice ultimately has little impact on how you'll write the code that sends and receives messages. The only significant differences will be in how you configure Spring to create connections to the broker.

RUNNING AN ARTEMIS BROKER You'll need an Artemis broker running to be able to run the code presented in this chapter. If you don't already have an Artemis instance running, you can follow the instructions from the Artemis documentation at **http://mng.bz/Xr81**.

By default, Spring assumes that your Artemis broker is listening on localhost at port 61616. That's fine for development purposes, but once you're ready to send your application into production, you'll need to set a few properties that tell Spring how to access the broker. The properties you'll find most useful are listed in table 9.1.

Table 9.1 Properties for configuring the location and credentials of an Artemis broker

Property	Description
spring.artemis.host	The broker's host
spring.artemis.port	The broker's port
spring.artemis.user	The user for accessing the broker (optional)
spring.artemis.password	The password for accessing the broker (optional)

For example, consider the following entry from an application.yml file that might be used in a nondevelopment setting:

```
spring:
  artemis:
    host: artemis.tacocloud.com
    port: 61617
    user: tacoweb
    password: l3tm31n
```

This sets up Spring to create broker connections to an Artemis broker listening at artemis.tacocloud.com, port 61617. It also sets the credentials for the application that will be interacting with that broker. The credentials are optional, but they're recommended for production deployments.

If you were to use ActiveMQ instead of Artemis, you'd need to use the ActiveMQ-specific properties listed in table 9.2.

Table 9.2 Properties for configuring the location and credentials of an ActiveMQ broker

Property	Description
spring.activemq.broker-url	The URL of the broker
spring.activemq.user	The user for accessing the broker (optional)
spring.activemq.password	The password for accessing the broker (optional)
spring.activemq.in-memory	Whether to start an in-memory broker (default: true)

Notice that instead of offering separate properties for the broker's hostname and port, an ActiveMQ broker's address is specified with a single property, `spring.activemq.broker-url`. The URL should be a tcp:// URL, as shown in the following YAML snippet:

```
spring:
  activemq:
    broker-url: tcp:/ /activemq.tacocloud.com
    user: tacoweb
    password: l3tm31n
```

Whether you choose Artemis or ActiveMQ, you shouldn't need to configure these properties for development when the broker is running locally.

If you're using ActiveMQ, you will, however, need to set the `spring.activemq.in-memory` property to `false` to prevent Spring from starting an in-memory broker. An in-memory broker may seem useful, but it's helpful only when you'll be consuming messages from the same application that publishes them (which has limited usefulness).

Instead of using an embedded broker, you'll want to install and start an Artemis (or ActiveMQ) broker before moving on. Rather than repeat the installation instructions here, I refer you to the broker documentation for details:

- *Artemis*—**http://mng.bz/yJOo**
- *ActiveMQ*—**http://mng.bz/MveD**

With the JMS starter in your build and a broker waiting to ferry messages from one application to another, you're ready to start sending messages.

1.1.2 Sending messages with JmsTemplate

With a JMS starter dependency (either Artemis or ActiveMQ) in your build, Spring Boot will autoconfigure a `JmsTemplate` (among other things) that you can inject and use to send and receive messages.

`JmsTemplate` is the centerpiece of Spring's JMS integration support. Much like Spring's other template-oriented components, `JmsTemplate` eliminates a lot of boilerplate code that would otherwise be required to work with JMS. Without `JmsTemplate`, you'd need to write code to create a connection and session with the message broker and more code to deal with any exceptions that might be thrown in the course of sending a message. `JmsTemplate` focuses on what you really want to do: send a message.

`JmsTemplate` has several methods that are useful for sending messages, including the following:

```
// Send raw messages
void send(MessageCreator messageCreator) throws JmsException;
void send(Destination destination, MessageCreator messageCreator)
                                    throws JmsException;
void send(String destinationName, MessageCreator messageCreator)
                                    throws JmsException;
// Send messages converted from objects
void convertAndSend(Object message) throws JmsException;
void convertAndSend(Destination destination, Object message)
                                    throws JmsException;
void convertAndSend(String destinationName, Object message)
                                    throws JmsException;

// Send messages converted from objects with post-processing
void convertAndSend(Object message,
        MessagePostProcessor postProcessor) throws JmsException;
void convertAndSend(Destination destination, Object message,
        MessagePostProcessor postProcessor) throws JmsException;
void convertAndSend(String destinationName, Object message,
        MessagePostProcessor postProcessor) throws JmsException;
```

As you can see, there are really only two methods, `send()` and `convertAndSend()`, each overridden to support different parameters. And if you look closer, you'll notice that the

various forms of `convertAndSend()` can be broken into two subcategories. In trying to understand what all of these methods do, consider the following breakdown:

- Three `send()` methods require a `MessageCreator` to manufacture a `Message` object.
- Three `convertAndSend()` methods accept an `Object` and automatically convert that `Object` into a `Message` behind the scenes.
- Three `convertAndSend()` methods automatically convert an `Object` to a `Message` but also accept a `MessagePostProcessor` to allow for customization of the `Message` before it's sent.

Moreover, each of these three method categories is composed of three overriding methods that are distinguished by how the JMS destination (queue or topic) is specified, as follows:

- One method accepts no destination parameter and sends the message to a default destination.
- One method accepts a `Destination` object that specifies the destination for the message.
- One method accepts a `String` that specifies the destination for the message by name.

Putting these methods to work, consider `JmsOrderMessagingService` in the next listing, which uses the most basic form of the `send()` method.

Listing 9.1 Sending an order with `.send()` to a default destination

```
package tacos.messaging;
import javax.jms.JMSException;
import javax.jms.Message;
import javax.jms.Session;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jms.core.JmsTemplate;
import org.springframework.jms.core.MessageCreator;
import org.springframework.stereotype.Service;

@Service
public class JmsOrderMessagingService implements OrderMessagingService {
  private JmsTemplate jms;

  @Autowired
  public JmsOrderMessagingService(JmsTemplate jms) {
```

```
    this.jms = jms;
  }

  @Override
  public void sendOrder(TacoOrder order) {
    jms.send(new MessageCreator() {
          @Override
          public Message createMessage(Session session)
                                  throws JMSException {
            return session.createObjectMessage(order);
          }
        }
    );
  }
}
```

The `sendOrder()` method calls `jms.send()`, passing in an anonymous inner-class implementation of `MessageCreator`. That implementation overrides `createMessage()` to create a new object message from the given `TacoOrder` object.

Because the JMS-specific `JmsOrderMessagingService` implements the more generic `OrderMessagingService` interface, we can put this service to work by injecting it into the `OrderApiController` and calling `sendOrder()` when an order is created, as shown here:

```
@RestController
@RequestMapping(path="/api/orders",
          produces="application/json")
@CrossOrigin(origins="http://localhost:8080")
public class OrderApiController {

  private OrderRepository repo;
  private OrderMessagingService messageService;

  public OrderApiController(
      OrderRepository repo,
      OrderMessagingService messageService) {
    this.repo = repo;
    this.messageService = messageService;
  }

  @PostMapping(consumes="application/json")
  @ResponseStatus(HttpStatus.CREATED)
  public TacoOrder postOrder(@RequestBody TacoOrder order) {
    messageService.sendOrder(order);
```

```
    return repo.save(order);
  }

  ...

}
```

Now when you create an order through the Taco Cloud website, a message should be sent to the broker for routing to another application that will receive the order. We don't yet have anything to receive that message, though. Even so, you can use the Artemis console to view the contents of the queue. See the Artemis documentation at http://mng.bz/aZx9 for details on how to do this.

I'm not sure about you, but I think the code in listing 9.1, although straightforward, is a bit clumsy. The ceremony involved in declaring an anonymous inner class complicates an otherwise simple method call. Recognizing that `MessageCreator` is a functional interface, you can tidy up the `sendOrder()` method a bit with a lambda, as shown next:

```
@Override
public void sendOrder(TacoOrder order) {
  jms.send(session -> session.createObjectMessage(order));
}
```

But notice that the call to `jms.send()` doesn't specify a destination. For this to work, you must also specify a default destination name with the `spring.jms.template .default-destination` property. For example, you could set the property in your application.yml file like this:

```
spring:
  jms:
    template:
      default-destination: tacocloud.order.queue
```

In many cases, using a default destination is the easiest choice. It lets you specify the destination name once, allowing the code to be concerned only with sending messages, without regard for where they're being sent. But if you ever need to send a message to a destination other than the default destination, you'll need to specify that destination as a parameter to `send()`.

One way of doing that is by passing a `Destination` object as the first parameter to `send()`. The easiest way to do this is to declare a `Destination` bean and then inject it into the bean that performs messaging. For example, the following bean declares the Taco Cloud order queue `Destination`:

```
@Bean
public Destination orderQueue() {
  return new ActiveMQQueue("tacocloud.order.queue");
}
```

This bean method can be added to any configuration class in the application that will be sending or receiving messages via JMS. For the sake of organization, it's best to add it to a configuration class designated for messaging configuration, such as `MessagingConfig`.

It's important to note that the `ActiveMQQueue` used here is actually from Artemis (from the `org.apache.activemq.artemis.jms.client` package). If you're using ActiveMQ (not Artemis), there's also a class named `ActiveMQQueue` (from the `org.apache.activemq.command` package).

If this `Destination` bean is injected into `JmsOrderMessagingService`, you can use it to specify the destination when calling `send()` as follows:

```
private Destination orderQueue;

@Autowired
public JmsOrderMessagingService(JmsTemplate jms,
                    Destination orderQueue) {
  this.jms = jms;
  this.orderQueue = orderQueue;
}

...

@Override
public void sendOrder(TacoOrder order) {
  jms.send(
      orderQueue,
      session -> session.createObjectMessage(order));
}
```

Specifying the destination with a `Destination` object like this affords you the opportunity to configure the `Destination` with more than just the destination name. But in practice, you'll almost never specify anything more than the destination name. It's often easier to just send the name as the first parameter to `send()`, as shown here:

```
@Override
public void sendOrder(TacoOrder order) {
 jms.send(
    "tacocloud.order.queue",
    session -> session.createObjectMessage(order));
}
```

Although the `send()` method isn't particularly difficult to use (especially when the `MessageCreator` is given as a lambda), a sliver of complexity is added by requiring that you provide a `MessageCreator`. Wouldn't it be simpler if you needed to specify only the object that's to be sent (and optionally the destination)? That describes succinctly how `convertAndSend()` works. Let's take a look.

CONVERTING MESSAGES BEFORE SENDING

The `JmsTemplatesconvertAndSend()` method simplifies message publication by eliminating the need to provide a `MessageCreator`. Instead, you pass the object that's to be sent directly to `convertAndSend()`, and the object will be converted into a `Message` before being sent.

For example, the following reimplementation of `sendOrder()` uses `convertAndSend()` to send a `TacoOrder` to a named destination:

```
@Override
public void sendOrder(TacoOrder order) {
 jms.convertAndSend("tacocloud.order.queue", order);
}
```

Just like the `send()` method, `convertAndSend()` will accept either a `Destination` or `String` value to specify the destination, or you can leave out the destination altogether to send the message to the default destination.

Whichever form of `convertAndSend()` you choose, the `TacoOrder` passed into `convertAndSend()` is converted into a `Message` before it's sent. Under the covers, this is

achieved with an implementation of `MessageConverter` that does the dirty work of converting application domain objects to `Message` objects.

CONFIGURING A MESSAGE CONVERTER

`MessageConverter` is a Spring-defined interface that has only the following two methods to be implemented:

```
public interface MessageConverter {
  Message toMessage(Object object, Session session)
              throws JMSException, MessageConversionException;
  Object fromMessage(Message message)
}
```

Although this interface is simple enough to implement, you often won't need to create a custom implementation. Spring already offers a handful of implementations, such as those described in table 9.3.

Table 9.3 Spring message converters for common conversion tasks (all in the `org.springframework.jms.support.converter` package)

Message converter	What it does
MappingJackson2MessageConverter	Uses the Jackson 2 JSON library to convert messages to and from JSON
`MarshallingMessageConverter`	Uses JAXB to convert messages to and from XML
MessagingMessageConverter	Converts a `Message` from the messaging abstraction to and from a `Message` using an underlying `MessageConverter` for the payload and a `JmsHeaderMapper` to map the JMS headers to and from standard message headers
`SimpleMessageConverter`	Converts a `String` to and from a `TextMessage`, `byte` arrays to and from a `BytesMessage`, a `Map` to and from a `MapMessage`, and a `Serializable` to and from an `ObjectMessage`

`SimpleMessageConverter` is the default, but it requires that the object being sent implement `Serializable`. This may be a good idea, but you may prefer to use one of the other message converters, such as `MappingJackson2MessageConverter`, to avoid that restriction.

To apply a different message converter, all you must do is declare an instance of the chosen converter as a bean. For example, the following bean declaration will enable `MappingJackson2MessageConverter` to be used instead of `SimpleMessageConverter`:

```
@Bean
public MappingJackson2MessageConverter messageConverter() {
  MappingJackson2MessageConverter messageConverter =
                    new MappingJackson2MessageConverter();
  messageConverter.setTypeIdPropertyName("_typeId");
  return messageConverter;
}
```

This bean method can be placed in any configuration class in the application that sends and receives messages with JMS, including alongside the `Destination` bean in `MessagingConfig`.

Notice that you called `setTypeIdPropertyName()` on the `MappingJackson2MessageConverter` before returning it. This is very important, because it enables the receiver to know what type to convert an incoming message to. By default, it will contain the fully qualified classname of the type being converted. But this method is somewhat inflexible, requiring that the receiver also have the same type, with the same fully qualified classname.

To allow for more flexibility, you can map a synthetic type name to the actual type by calling `setTypeIdMappings()` on the message converter. For example, the following change to the message converter bean method maps a synthetic `TacoOrder` type ID to the `TacoOrder` class:

```
@Bean
public MappingJackson2MessageConverter messageConverter() {
  MappingJackson2MessageConverter messageConverter =
                      new MappingJackson2MessageConverter();
  messageConverter.setTypeIdPropertyName("_typeId");

  Map<String, Class<?>> typeIdMappings = new HashMap<String, Class<?>>();
  typeIdMappings.put("order", TacoOrder.class);
  messageConverter.setTypeIdMappings(typeIdMappings);

  return messageConverter;
}
```

Instead of the fully qualified classname being sent in the message's `_typeId` property, the value `TacoOrder` will be sent. In the receiving application, a similar message converter will have been configured, mapping `TacoOrder` to its own understanding of what an order is. That implementation of an order may be in a different package, have a different name, and even have a subset of the sender's `TacoOrder` properties.

POSTPROCESSING MESSAGES

Let's suppose that in addition to its lucrative web business, Taco Cloud has decided to open a few brick-and-mortar taco joints. Given that any of their restaurants could also be a fulfillment center for the web business, they need a way to communicate the source of

an order to the kitchens at the restaurants. This will enable the kitchen staff to employ a different process for store orders than for web orders.

It would be reasonable to add a new `source` property to the `TacoOrder` object to carry this information, populating it with `WEB` for orders placed online and with `STORE` for orders placed in the stores. But that would require a change to both the website's `TacoOrder` class and the kitchen application's `TacoOrder` class when, in reality, it's information that's required only for the taco preparers.

An easier solution would be to add a custom header to the message to carry the order's source. If you were using the `send()` method to send the taco orders, this could easily be accomplished by calling `setStringProperty()` on the `Message` object as follows:

```
jms.send("tacocloud.order.queue",
    session -> {
      Message message = session.createObjectMessage(order);
      message.setStringProperty("X_ORDER_SOURCE", "WEB");
    });
```

The problem here is that you aren't using `send()`. By choosing to use `convertAndSend()`, the `Message` object is created under the covers, and you don't have access to it.

Fortunately, you have a way to tweak a `Message` created under the covers before it's sent. By passing in a `MessagePostProcessor` as the final parameter to `convertAndSend()`, you can do whatever you want with the `Message` after it has been created. The following code still uses `convertAndSend()`, but it also uses a `MessagePostProcessor` to add the `X_ORDER_SOURCE` header before the message is sent:

```
jms.convertAndSend("tacocloud.order.queue", order, new MessagePostProcessor() {
  @Override
  public Message postProcessMessage(Message message) throws JMSException {
    message.setStringProperty("X_ORDER_SOURCE", "WEB");
    return message;
  }
});
```

You may have noticed that `MessagePostProcessor` is a functional interface. This means that you can simplify it a bit by replacing the anonymous inner class with a lambda as shown here:

```
jms.convertAndSend("tacocloud.order.queue", order,
    message -> {
      message.setStringProperty("X_ORDER_SOURCE", "WEB");
      return message;
    });
```

Although you need this particular `MessagePostProcessor` for only this one call to `convertAndSend()`, you may find yourself using the same `MessagePostProcessor` for several different calls to `convertAndSend()`. In those cases, perhaps a method reference, shown next, is a better choice than a lambda, avoiding unnecessary code duplication:

```
@GetMapping("/convertAndSend/order")
public String convertAndSendOrder() {
  TacoOrder order = buildOrder();
  jms.convertAndSend("tacocloud.order.queue", order,
      this::addOrderSource);
  return "Convert and sent order";
}

private Message addOrderSource(Message message) throws JMSException {
  message.setStringProperty("X_ORDER_SOURCE", "WEB");
  return message;
}
```

You've now seen several ways of sending messages. But it does no good to send a message if nobody ever receives it. Let's look at how you can receive messages with Spring JMS.

1.3 Receiving JMS messages

When it comes to consuming messages, you have the choice of a *pull model*, where your code requests a message and waits until one arrives, or a *push model*, in which messages are handed to your code as they become available.

`JmsTemplate` offers several methods for receiving messages, but all of them use a pull model. You call one of those methods to request a message, and the thread is blocked until a message is available (which could be immediately or it might take a while).

On the other hand, you also have the option of using a push model, wherein you define a message listener that's invoked any time a message is available.

Both options are suitable for a variety of use cases. It's generally accepted that the push model is the best choice, because it doesn't block a thread. But in some use cases, a listener could be overburdened if messages arrive too quickly. The pull model enables a consumer to declare that they're ready to process a new message.

Let's look at both ways of receiving messages. We'll start with the pull model offered by `JmsTemplate`.

RECEIVING WITH JMSTEMPLATE

`JmsTemplate` offers several methods for pulling methods from the broker, including the following:

```
Message receive() throws JmsException;
Message receive(Destination destination) throws JmsException;
Message receive(String destinationName) throws JmsException;

Object receiveAndConvert() throws JmsException;
Object receiveAndConvert(Destination destination) throws JmsException;
Object receiveAndConvert(String destinationName) throws JmsException;
```

As you can see, these six methods mirror the `send()` and `convertAndSend()` methods from `JmsTemplate`. The `receive()` methods receive a raw `Message`, whereas the `receiveAndConvert()` methods use a configured message converter to convert messages into domain types. And for each of these, you can specify either a `Destination` or a `String` containing the destination name, or you can pull a message from the default destination.

To see these in action, you'll write some code that pulls an `TacoOrder` from the `tacocloud.order.queue` destination. The following listing shows `OrderReceiver`, a service component that receives order data using `JmsTemplate.receive()`.

Listing 9.2 Pulling orders from a queue

```
package tacos.kitchen.messaging.jms;
import javax.jms.Message;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jms.core.JmsTemplate;
import org.springframework.jms.support.converter.MessageConverter;
import org.springframework.stereotype.Component;
```

```
@Component
public class JmsOrderReceiver implements OrderReceiver {
  private JmsTemplate jms;
  private MessageConverter converter;

  @Autowired
  public JmsOrderReceiver(JmsTemplate jms, MessageConverter converter) {
    this.jms = jms;
    this.converter = converter;
  }
  public TacoOrder receiveOrder() {
    Message message = jms.receive("tacocloud.order.queue");
    return (TacoOrder) converter.fromMessage(message);
  }
}
```

Here you've used a `String` to specify the destination from which to pull an order. The `receive()` method returns an unconverted `Message`. But what you really need is the `TacoOrder` that's inside of the `Message`, so the very next thing that happens is that you use an injected message converter to convert the message. The type ID property in the message will guide the converter in converting it to a `TacoOrder`, but it's returned as an `Object` that requires casting before you can return it.

Receiving a raw `Message` object might be useful in some cases where you need to inspect the message's properties and headers. But often you need only the payload. Converting that payload to a domain type is a two-step process and requires that the message converter be injected into the component. When you care only about the message's payload, `receiveAndConvert()` is a lot simpler. The next listing shows how `JmsOrderReceiver` could be reworked to use `receiveAndConvert()` instead of `receive()`.

Listing 9.3 Receiving an already-converted `TacoOrder` object

```
package tacos.kitchen.messaging.jms;

import org.springframework.jms.core.JmsTemplate;
import org.springframework.stereotype.Component;
import tacos.TacoOrder;
import tacos.kitchen.OrderReceiver;

@Component
public class JmsOrderReceiver implements OrderReceiver {

  private JmsTemplate jms;
```

```
  public JmsOrderReceiver(JmsTemplate jms) {
    this.jms = jms;
  }

  @Override
  public TacoOrder receiveOrder() {
    return (TacoOrder) jms.receiveAndConvert("tacocloud.order.queue");
  }

}
```

This new version of `JmsOrderReceiver` has a `receiveOrder()` method that has been reduced to only one line. Plus, you no longer need to inject a `MessageConverter`, because all of the message conversion will be done behind the scenes in `receiveAndConvert()`.

Before moving on, let's consider how `receiveOrder()` might be used in the Taco Cloud kitchen application. A food preparer at one of Taco Cloud's kitchens might push a button or take some action to indicate that they're ready to start building tacos. At that point, `receiveOrder()` would be invoked and the call to `receive()` or `receiveAndConvert()` would block. Nothing else would happen until an order message is ready. Once an order arrives, it will be returned from `receiveOrder()`, and its information will be used to display the details of the order for the food preparer to get to work. This seems like a natural choice for a pull model.

Now let's see how a push model works by declaring a JMS listener.

DECLARING MESSAGE LISTENERS

Unlike the pull model, where an explicit call to `receive()` or `receiveAndConvert()` was required to receive a message, a message listener is a passive component that's idle until a message arrives.

To create a message listener that reacts to JMS messages, you simply annotate a method in a component with `@JmsListener`. The next listing shows a new `OrderListener` component that listens passively for messages, rather than actively requesting them.

Listing 9.4 An `OrderListener` component that listens for orders

```
package tacos.kitchen.messaging.jms.listener;

import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.context.annotation.Profile;
import org.springframework.jms.annotation.JmsListener;
import org.springframework.stereotype.Component;

import tacos.TacoOrder;
import tacos.kitchen.KitchenUI;

@Profile("jms-listener")
@Component
public class OrderListener {

  private KitchenUI ui;

  @Autowired
  public OrderListener(KitchenUI ui) {
    this.ui = ui;
  }

  @JmsListener(destination = "tacocloud.order.queue")
  public void receiveOrder(TacoOrder order) {
    ui.displayOrder(order);
  }

}
```

The `receiveOrder()` method is annotated with `JmsListener` to "listen" for messages on the `tacocloud.order.queue` destination. It doesn't deal with `JmsTemplate`, nor is it explicitly invoked by your application code. Instead, framework code within Spring waits for messages to arrive on the specified destination, and when they arrive, the `receiveOrder()` method is invoked automatically with the message's `TacoOrder` payload as a parameter.

In many ways, the `@JmsListener` annotation is like one of Spring MVC's request mapping annotations, such as `@GetMapping` or `@PostMapping`. In Spring MVC, methods annotated with one of the request mapping methods react to requests to a specified path. Similarly, methods that are annotated with `@JmsListener` react to messages that arrive in a destination.

Message listeners are often touted as the best choice because they don't block and are able to handle multiple messages quickly. In the context of the Taco Cloud application, however, perhaps they aren't the best choice. The food preparers are a significant bottleneck in the system and may not be able to prepare tacos as quickly as orders come in. A food preparer may have half-fulfilled an order when a new order is displayed on the

screen. The kitchen user interface would need to buffer the orders as they arrive to avoid overburdening the kitchen staff.

That's not to say that message listeners are bad. On the contrary, they're a perfect fit when messages can be handled quickly. But when the message handlers need to be able to ask for more messages on their own timing, the pull model offered by `JmsTemplate` seems more fitting.

Because JMS is defined by a standard Java specification and supported by many message broker implementations, it's a common choice for messaging in Java. But JMS has a few shortcomings, not the least of which is that as a Java specification, its use is limited to Java applications. Newer messaging options such as RabbitMQ and Kafka address these shortcomings and are available for other languages and platforms beyond the JVM. Let's set JMS aside and see how you could have implemented your taco order messaging with RabbitMQ.

9.2 Working with RabbitMQ and AMQP

As arguably the most prominent implementation of AMQP, RabbitMQ offers a more advanced message-routing strategy than JMS. Whereas JMS messages are addressed with the name of a destination from which the receiver will retrieve them, AMQP messages are addressed with the name of an exchange and a routing key, which are decoupled from the queue to which the receiver is listening. This relationship between an exchange and queues is illustrated in figure 9.2.

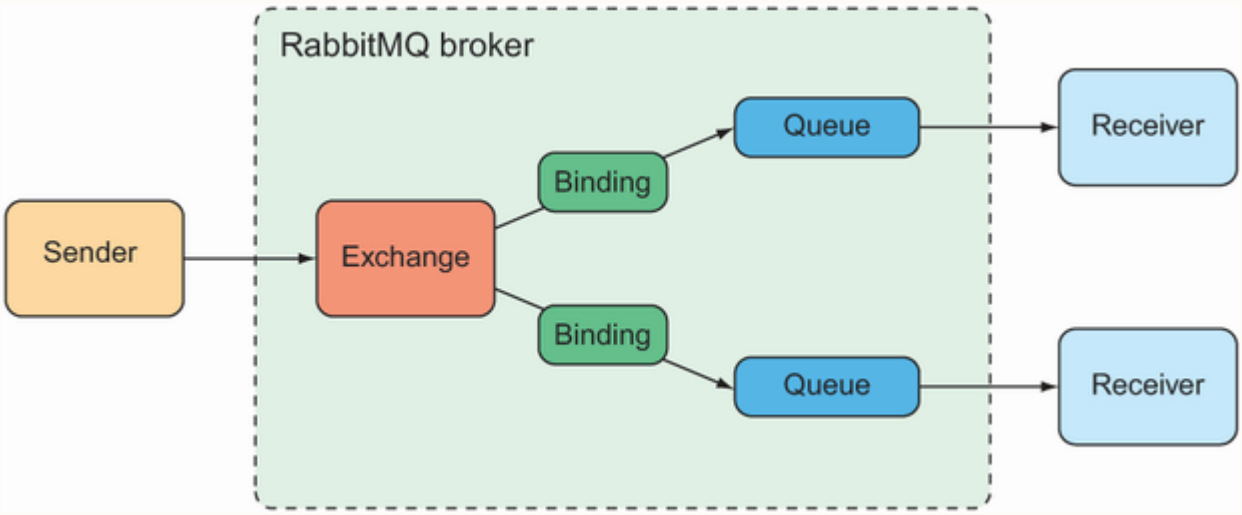


Figure 9.2 Messages sent to a RabbitMQ exchange are routed to one or more queues, based on routing keys and bindings.

When a message arrives at the RabbitMQ broker, it goes to the exchange for which it was addressed. The exchange is responsible for routing it to one or more queues, depending

on the type of exchange, the binding between the exchange and queues, and the value of the message's routing key.

There are several different kinds of exchanges, including the following:

- *Default*—A special exchange that's automatically created by the broker. It routes messages to queues whose name is the same as the message's routing key. All queues will automatically be bound to the default exchange.
- *Direct*—Routes messages to a queue whose binding key is the same as the message's routing key.
- *Topic*—Routes a message to one or more queues where the binding key (which may contain wildcards) matches the message's routing key.
- *Fanout*—Routes messages to all bound queues without regard for binding keys or routing keys.
- *Headers*—Similar to a topic exchange, except that routing is based on message header values rather than routing keys.
- *Dead letter*—A catchall for any messages that are undeliverable (meaning they don't match any defined exchange-to-queue binding).

The simplest forms of exchanges are default and fanout—these roughly correspond to a JMS queue and topic. But the other exchanges allow you to define more flexible routing schemes.

The most important thing to understand is that messages are sent to exchanges with routing keys and they're consumed from queues. How they get from an exchange to a queue depends on the binding definitions and what best suits your use cases.

Which exchange type you use and how you define the bindings from exchanges to queues has little bearing on how messages are sent and received in your Spring applications. Therefore, we'll focus on how to write code that sends and receives messages with Rabbit.

NOTE For a more detailed discussion on how best to bind queues to exchanges, see *RabbitMQ in Depth* by Gavin Roy (Manning, 2017) or *RabbitMQ in Action* by Alvaro Videla and Jason J. W. Williams (Manning, 2012).

2.2.1 Adding RabbitMQ to Spring

Before you can start sending and receiving RabbitMQ messages with Spring, you'll need to add Spring Boot's AMQP starter dependency to your build in place of the Artemis or ActiveMQ starter you added in the previous section, as shown here:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
```

Adding the AMQP starter to your build will trigger autoconfiguration that will create an AMQP connection factory and `RabbitTemplate` beans, as well as other supporting components. Simply adding this dependency is all you need to do to start sending and receiving messages from a RabbitMQ broker with Spring. But there are a handful of useful properties you'll want to know about, listed in table 9.4.

Table 9.4 Properties for configuring the location and credentials of a RabbitMQ broker

Property	Description
spring.rabbitmq.addresses	A comma-separated list of RabbitMQ broker addresses
spring.rabbitmq.host	The broker's host (defaults to localhost)
spring.rabbitmq.port	The broker's port (defaults to 5672)
spring.rabbitmq.username	The username for accessing the broker (optional)
spring.rabbitmq.password	The password for accessing the broker (optional)

For development purposes, you'll probably have a RabbitMQ broker that doesn't require authentication running on your local machine, listening on port 5672. These properties likely won't get much use while you're still in development, but they'll no doubt prove useful when your applications move into production.

RUNNING A RABBITMQ BROKER If you don't already have a RabbitMQ broker to work with, you have several options for running RabbitMQ on your local machine. See the official RabbitMQ documentation at **https://www.rabbitmq .com/download.html** for the latest instructions for running RabbitMQ.

For example, suppose that as you move into production, your RabbitMQ broker is on a server named rabbit.tacocloud.com, listening on port 5673, and requiring credentials. In that case, the following configuration in your application.yml file will set those properties when the `prod` profile is active:

```
spring:
  profiles: prod
  rabbitmq:
    host: rabbit.tacocloud.com
    port: 5673
    username: tacoweb
    password: l3tm31n
```

Now that RabbitMQ is configured in your application, it's time to start sending messages with `RabbitTemplate`.

2.2.2 Sending messages with RabbitTemplate

At the core of Spring's support for RabbitMQ messaging is `RabbitTemplate`. `RabbitTemplate` is similar to `JmsTemplate` and offers a similar set of methods. As you'll see, however, some subtle differences align with the unique way that RabbitMQ works.

With regard to sending messages with `RabbitTemplate`, the `send()` and `convertAndSend()` methods parallel the same-named methods from `JmsTemplate`. But unlike the `JmsTemplate` methods, which route messages only to a given queue or topic, `RabbitTemplate` methods send messages in terms of exchanges and routing keys. Here are a few of the most relevant methods for sending messages with `RabbitTemplate`:[1]

```
// Send raw messages
void send(Message message) throws AmqpException;
void send(String routingKey, Message message) throws AmqpException;
void send(String exchange, String routingKey, Message message)
                    throws AmqpException;

// Send messages converted from objects
void convertAndSend(Object message) throws AmqpException;
void convertAndSend(String routingKey, Object message)
                    throws AmqpException;
void convertAndSend(String exchange, String routingKey,
                Object message) throws AmqpException;

// Send messages converted from objects with post-processing
void convertAndSend(Object message, MessagePostProcessor mPP)
                    throws AmqpException;
void convertAndSend(String routingKey, Object message,
                MessagePostProcessor messagePostProcessor)
                throws AmqpException;
```

```
void convertAndSend(String exchange, String routingKey,
                    Object message,
                    MessagePostProcessor messagePostProcessor)
                    throws AmqpException;
```

As you can see, these methods follow a pattern similar to their twins in `JmsTemplate`. The first three `send()` methods all send a raw `Message` object. The next three `convertAndSend()` methods accept an object that will be converted to a `Message` behind the scenes before being sent. The final three `convertAndSend()` methods are like the previous three, but they accept a `MessagePostProcessor` that can be used to manipulate the `Message` object before it's sent to the broker.

These methods differ from their `JmsTemplate` counterparts in that they accept `String` values to specify an exchange and routing key, rather than a destination name (or `Destination` object). The methods that don't take an exchange will have their messages sent to the default exchange. Likewise, the methods that don't take a routing key will have their messages routed with a default routing key.

Let's put `RabbitTemplate` to work sending taco orders. One way you can do that is by using the `send()` method, as shown in listing 9.5. But before you can call `send()`, you'll need to convert a `TacoOrder` object to a `Message`. That could be a tedious job, if not for the fact that `RabbitTemplate` makes its message converter readily available with a `getMessageConverter()` method.

Listing 9.5 Sending a message with `RabbitTemplate.send()`

```
package tacos.messaging;
import org.springframework.amqp.core.Message;
import org.springframework.amqp.core.MessageProperties;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
import
  org.springframework.amqp.support.converter.MessageConverter;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import tacos.Order;

@Service
public class RabbitOrderMessagingService
      implements OrderMessagingService {
 private RabbitTemplate rabbit;

 @Autowired
```

```
  public RabbitOrderMessagingService(RabbitTemplate rabbit) {
    this.rabbit = rabbit;
  }

  public void sendOrder(TacoOrder order) {
    MessageConverter converter = rabbit.getMessageConverter();
    MessageProperties props = new MessageProperties();
    Message message = converter.toMessage(order, props);
    rabbit.send("tacocloud.order", message);
  }
}
```

You'll notice that `RabbitOrderMessagingService` implements `OrderMessagingService`, just like `JmsOrderMessagingService`. This means that it can be injected into `OrderApiController` the same way to send order messages when an order is placed. Because we don't yet have anything to receive those messages, though, you can use the RabbitMQ browser-based management console. See **https://www.rabbitmq.com/management.html** for details on how to enable and use the RabbitMQ console.

Once you have a `MessageConverter` in hand, it's simple work to convert a `TacoOrder` to a `Message`. You must supply any message properties with a `MessageProperties`, but if you don't need to set any such properties, a default instance of `MessageProperties` is fine. Then, all that's left is to call `send()`, passing in the exchange and routing key (both of which are optional) along with the message. In this example, you're specifying only the routing key—`tacocloud.order`—along with the message, so the default exchange will be used.

Speaking of default exchanges, the default exchange name is `""` (an empty `String`), which corresponds to the default exchange that's automatically created by the RabbitMQ broker. Likewise, the default routing key is `""` (whose routing is dependent upon the exchange and bindings in question). You can override these defaults by setting the `spring.rabbitmq.template.exchange` and `spring.rabbitmq.template .routing-key` properties as follows:

```
spring:
  rabbitmq:
    template:
      exchange: tacocloud.order
      routing-key: kitchens.central
```

In this case, all messages sent without specifying an exchange will automatically be sent to the exchange whose name is `tacocloud.order`. If the routing key is also unspecified in the call to `send()` or `convertAndSend()`, the messages will have a routing key of `kitchens.central`.

Creating a `Message` object from the message converter is easy enough, but it's even easier to use `convertAndSend()` to let `RabbitTemplate` handle all of the conversion work for you, as shown next:

```
public void sendOrder(TacoOrder order) {
  rabbit.convertAndSend("tacocloud.order", order);
}
```

CONFIGURING A MESSAGE CONVERTER

By default, message conversion is performed with `SimpleMessageConverter`, which is able to convert simple types (like `String`) and `Serializable` objects to `Message` objects. But Spring offers several message converters for `RabbitTemplate`, including the following:

- `Jackson2JsonMessageConverter`—Converts objects to and from JSON using the Jackson 2 JSON processor
- `MarshallingMessageConverter`—Converts using a Spring `Marshaller` and `Unmarshaller`
- `SerializerMessageConverter`—Converts `String` and native objects of any kind using Spring's `Serializer` and `Deserializer` abstractions
- `SimpleMessageConverter`—Converts `String`, `byte` arrays, and `Serializable` types
- `ContentTypeDelegatingMessageConverter`—Delegates to another `MessageConverter` based on the `contentType` header
- `MessagingMessageConverter`—Delegates to an underlying `MessageConverter` for the message conversion and to an `AmqpHeaderConverter` for the headers

If you need to change the message converter, just configure a bean of type `MessageConverter`. For example, for JSON-based message conversion, you can configure a `Jackson2JsonMessageConverter` like this:

```
@Bean
public Jackson2JsonMessageConverter messageConverter() {
  return new Jackson2JsonMessageConverter();
}
```

Spring Boot autoconfiguration will discover this bean and inject it into `RabbitTemplate` in place of the default message converter.

SETTING MESSAGE PROPERTIES

As with JMS, you may need to set some headers in the messages you send. For example, let's say you need to send an `X_ORDER_SOURCE` for all orders submitted through the Taco Cloud website. When creating your own `Message` objects, you can set the header through the `MessageProperties` instance you give to the message converter. Revisiting the `sendOrder()` method from listing 9.5, you only need one additional line of code to set the header, as shown next:

```
public void sendOrder(TacoOrder order) {
  MessageConverter converter = rabbit.getMessageConverter();
  MessageProperties props = new MessageProperties();
  props.setHeader("X_ORDER_SOURCE", "WEB");
  Message message = converter.toMessage(order, props);
  rabbit.send("tacocloud.order", message);
}
```

When using `convertAndSend()`, however, you don't have quick access to the `MessageProperties` object. A `MessagePostProcessor` can help you with that, though, as shown here:

```
public void sendOrder(TacoOrder order) {
   rabbit.convertAndSend("tacocloud.order.queue", order,
      new MessagePostProcessor() {
       @Override
       public Message postProcessMessage(Message message)
          throws AmqpException {
         MessageProperties props = message.getMessageProperties();
         props.setHeader("X_ORDER_SOURCE", "WEB");
         return message;
       }
      });
 }
```

Here you supply `convertAndSend()` with an anonymous inner-class implementation of `MessagePostProcessor`. In the `postProcessMessage()` method, you pull the `MessageProperties` from the `Message` and then call `setHeader()` to set the `X_ORDER_SOURCE` header.

Now that you've seen how to send messages with `RabbitTemplate`, let's switch our focus over to the code that receives messages from a RabbitMQ queue.

2.3 Receiving messages from RabbitMQ

You've seen that sending messages with `RabbitTemplate` doesn't differ much from sending messages with `JmsTemplate`. And as it turns out, receiving messages from a RabbitMQ queue isn't very different than from JMS.

As with JMS, you have the following two choices:

- Pulling messages from a queue with `RabbitTemplate`
- Having messages pushed to a `@RabbitListener`-annotated method

Let's start by looking at the pull-based `RabbitTemplate.receive()` method.

RECEIVING MESSAGES WITH RABBITTEMPLATE

`RabbitTemplate` comes with several methods for pulling messages from a queue. A few of the most useful ones are listed here:

```
// Receive messages
Message receive() throws AmqpException;
Message receive(String queueName) throws AmqpException;
Message receive(long timeoutMillis) throws AmqpException;
Message receive(String queueName, long timeoutMillis) throws AmqpException;

// Receive objects converted from messages
Object receiveAndConvert() throws AmqpException;
Object receiveAndConvert(String queueName) throws AmqpException;
Object receiveAndConvert(long timeoutMillis) throws AmqpException;
Object receiveAndConvert(String queueName, long timeoutMillis)
                                    throws AmqpException;

// Receive type-safe objects converted from messages
<T> T receiveAndConvert(ParameterizedTypeReference<T> type)
                                    throws AmqpException;
<T> T receiveAndConvert(
   String queueName, ParameterizedTypeReference<T> type)
                                    throws AmqpException;
<T> T receiveAndConvert(
   long timeoutMillis, ParameterizedTypeReference<T> type)
                                    throws AmqpException;
<T> T receiveAndConvert(String queueName, long timeoutMillis,
```

```
    ParameterizedTypeReference<T> type)
                                    throws AmqpException;
```

These methods are the mirror images of the `send()` and `convertAndSend()` methods described earlier. Whereas `send()` is used to send raw `Message` objects, `receive()` receives raw `Message` objects from a queue. Likewise, `receiveAndConvert()` receives messages and uses a message converter to convert them into domain objects before returning them.

But a few obvious differences occur in the method signatures. First, none of these methods take an exchange or routing key as a parameter. That's because exchanges and routing keys are used to route messages to queues, but once the messages are in the queue, their next destination is the consumer who pulls them off the queue. Consuming applications needn't concern themselves with exchanges or routing keys. A queue is the only thing the consuming applications need to know about.

You'll also notice that many of the methods accept a `long` parameter to indicate a time-out for receiving the messages. By default, the receive time-out is 0 milliseconds. That is, a call to `receive()` will return immediately, potentially with a `null` value if no messages are available. This is a marked difference from how the `receive()` methods behave in `JmsTemplate`. By passing in a time-out value, you can have the `receive()` and `receiveAndConvert()` methods block until a message arrives or until the time-out expires. But even with a non-zero time-out, your code will need to be ready to deal with a `null` return.

Let's see how you can put this in action. The next listing shows a new Rabbit-based implementation of `OrderReceiver` that uses `RabbitTemplate` to receive orders.

Listing 9.6 Pulling orders from RabbitMQ with `RabbitTemplate`

```
package tacos.kitchen.messaging.rabbit;
import org.springframework.amqp.core.Message;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.amqp.support.converter.MessageConverter;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component
public class RabbitOrderReceiver {
  private RabbitTemplate rabbit;
  private MessageConverter converter;
```

```
  @Autowired
  public RabbitOrderReceiver(RabbitTemplate rabbit) {
    this.rabbit = rabbit;
    this.converter = rabbit.getMessageConverter();
  }

  public TacoOrder receiveOrder() {
    Message message = rabbit.receive("tacocloud.order");
    return message != null
        ? (TacoOrder) converter.fromMessage(message)
        : null;
  }
}
```

The `receiveOrder()` method is where all of the action takes place. It makes a call to the `receive()` method on the injected `RabbitTemplate` to pull an order from the `tacocloud.order` queue. It provides no time-out value, so you can assume only that the call returns immediately with either a `Message` or `null`. If a `Message` is returned, you use the `MessageConverter` from the `RabbitTemplate` to convert the `Message` to a `TacoOrder`. On the other hand, if `receive()` returns `null`, you'll return a `null`.

Depending on the use case, you may be able to tolerate a small delay. In the Taco Cloud kitchen's overhead display, for example, you can possibly wait a while if no orders are available. Let's say you decide to wait up to 30 seconds before giving up. Then the `receiveOrder()` method can be changed to pass a 30,000 millisecond delay to `receive()` as follows:

```
public TacoOrder receiveOrder() {
  Message message = rabbit.receive("tacocloud.order.queue", 30000);
  return message != null
      ? (TacoOrder) converter.fromMessage(message)
      : null;
}
```

If you're like me, seeing a hardcoded number like that gives you a bit of discomfort. You might be thinking that it'd be a good idea to create a `@ConfigurationProperties`-annotated class so you could configure that time-out with a Spring Boot configuration property. I'd agree with you, if it weren't for the fact that Spring Boot already offers such a configuration property. If you want to set the time-out via configuration, simply remove

the time-out value in the call to `receive()` and set it in your configuration with the `spring.rabbitmq.template.receive-timeout` property like so:

```
spring:
  rabbitmq:
    template:
      receive-timeout: 30000
```

Back in the `receiveOrder()` method, notice that you had to use the message converter from `RabbitTemplate` to convert the incoming `Message` object to a `TacoOrder` object. But if the `RabbitTemplate` is carrying a message converter around, why can't it do the conversion for you? That's precisely what the `receiveAndConvert()` method is for. Using `receiveAndConvert()`, you can rewrite `receiveOrder()` like this:

```
public TacoOrder receiveOrder() {
  return (TacoOrder) rabbit.receiveAndConvert("tacocloud.order.queue");
}
```

That's a lot simpler, isn't it? The only troubling thing I see is the cast from `Object` to `TacoOrder`. There's an alternative to casting, though. Instead, you can pass a `ParameterizedTypeReference` to `receiveAndConvert()` to receive a `TacoOrder` object directly as follows:

```
public TacoOrder receiveOrder() {
  return rabbit.receiveAndConvert("tacocloud.order.queue",
              new ParameterizedTypeReference<Order>() {});
}
```

It's debatable whether that's better than casting, but it is a more type-safe approach than casting. The only requirement to using a `ParameterizedTypeReference` with `receiveAndConvert()` is that the message converter must be an implementation of `SmartMessageConverter`; `Jackson2JsonMessageConverter` is the only out-of-the-box implementation to choose from.

The pull model offered by `JmsTemplate` fits a lot of use cases, but often it's better to have code that listens for messages and that's invoked when messages arrive. Let's see how you can write message-driven beans that respond to RabbitMQ messages.

HANDLING RABBITMQ MESSAGES WITH LISTENERS

For message-driven RabbitMQ beans, Spring offers `RabbitListener`, the RabbitMQ counterpart to `JmsListener`. To specify that a method should be invoked when a message arrives in a RabbitMQ queue, annotate a bean's method with `@RabbitListener`.

For example, the following listing shows a RabbitMQ implementation of `OrderReceiver` that's annotated to listen for order messages rather than to poll for them with `RabbitTemplate`.

Listing 9.7 Declaring a method as a RabbitMQ message listener

```
package tacos.kitchen.messaging.rabbit.listener;

import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
import tacos.TacoOrder;
import tacos.kitchen.KitchenUI;

@Component
public class OrderListener {

  private KitchenUI ui;

  @Autowired
  public OrderListener(KitchenUI ui) {
    this.ui = ui;
  }

  @RabbitListener(queues = "tacocloud.order.queue")
  public void receiveOrder(TacoOrder order) {
    ui.displayOrder(order);
  }

}
```

You'll no doubt notice that this looks remarkably like the code from listing 9.4. Indeed, the only thing that changed was the listener annotation—from `@JmsListener` to `@RabbitListener`. As wonderful as `@RabbitListener` is, this near duplication of code leaves me with little to say about `@RabbitListener` that I haven't already said about `@JmsListener`. They're both great for writing code that responds to messages that are

pushed to them from their respective brokers—a JMS broker for `@JmsListener` and a RabbitMQ broker for `@RabbitListener`.

Although you may sense a lack of enthusiasm about `@RabbitListener` in that previous paragraph, be certain that isn't my intent. In truth, the fact that `@RabbitListener` works much like `@JmsListener` is actually quite exciting! It means you don't need to learn a completely different programming model when working with RabbitMQ vs. Artemis or ActiveMQ. The same excitement holds true for the similarities between `RabbitTemplate` and `JmsTemplate`.

Let's hold on to that excitement as we wrap up this chapter by looking at one more messaging option supported by Spring: Apache Kafka.

9.3 Messaging with Kafka

Apache is the newest messaging option we're examining in this chapter. At a glance, Kafka is a message broker just like ActiveMQ, Artemis, or Rabbit. But Kafka has a few unique tricks up its sleeves.

Kafka is designed to run in a cluster, affording great scalability. And by partitioning its topics across all instances in the cluster, it's very resilient. Whereas RabbitMQ deals primarily with queues in exchanges, Kafka utilizes topics only to offer pub/sub messaging.

Kafka topics are replicated across all brokers in the cluster. Each node in the cluster acts as a leader for one or more topics, being responsible for that topic's data and replicating it to the other nodes in the cluster.

Going a step further, each topic can be split into multiple partitions. In that case, each node in the cluster is the leader for one or more partitions of a topic, but not for the entire topic. Responsibility for the topic is split across all nodes. Figure 9.3 illustrates how this works.

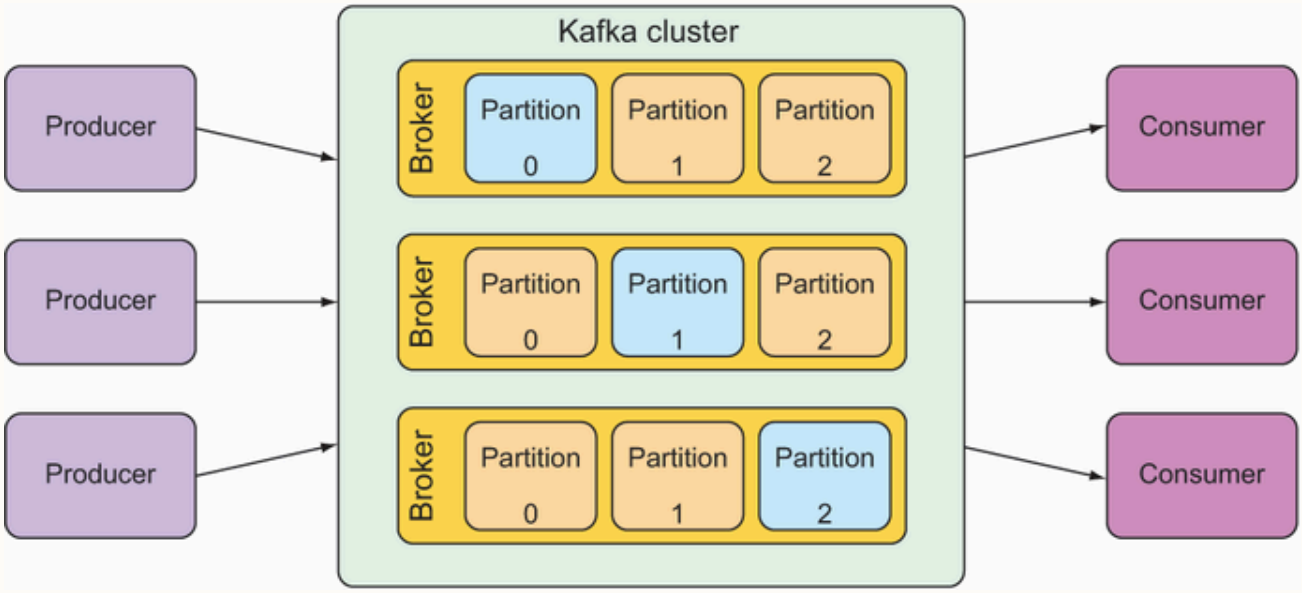


Figure 9.3 A Kafka cluster is composed of multiple brokers, each acting as a leader for partitions of the topics.

Due to Kafka's unique architecture, I encourage you to read more about it in *Kafka in Action* by Dylan Scott, Viktor Gamov, and Dave Klein (Manning, 2021). For our purposes, we'll focus on how to send messages to and receive them from Kafka with Spring.

3.1 Setting up Spring for Kafka messaging

To start using Kafka for messaging, you'll need to add the appropriate dependencies to your build. Unlike the JMS and RabbitMQ options, however, there isn't a Spring Boot starter for Kafka. Have no fear, though; you'll only need one dependency, shown next:

```
<dependency>
        <groupId>org.springframework.kafka</groupId>
        <artifactId>spring-kafka</artifactId>
    </dependency>
```

This one dependency brings everything you need for Kafka to the project. What's more, its presence will trigger Spring Boot autoconfiguration for Kafka that will, among other things, arrange for a `KafkaTemplate` in the Spring application context. All you need to do is inject the `KafkaTemplate` and go to work sending and receiving messages.

Before you start sending and receiving messages, however, you should be aware of a few properties that will come in handy when working with Kafka. Specifically, `KafkaTemplate` defaults to work with a Kafka broker on localhost, listening on port 9092. It's fine to start up a Kafka broker locally while developing an application, but when it's time to go to production, you'll need to configure a different host and port.

INSTALLING A KAFKA CLUSTER You'll need a Kafka cluster available if you want to run the examples presented in this chapter. The Kafka documentation at https://kafka.apache.org/quickstart is a great place to start to learn how to run Kafka locally on your machine.

The `spring.kafka.bootstrap-servers` property sets the location of one or more Kafka servers used to establish an initial connection to the Kafka cluster. For example, if one of the Kafka servers in the cluster is running at kafka.tacocloud.com and listening on port 9092, you can configure its location in YAML like this:

```
spring:
  kafka:
    bootstrap-servers:
    - kafka.tacocloud.com:9092
```

But notice that `spring.kafka.bootstrap-servers` is plural and accepts a list. As such, you can provide it with multiple Kafka servers in the cluster, as shown next:

```
spring:
  kafka:
    bootstrap-servers:
    - kafka.tacocloud.com:9092
    - kafka.tacocloud.com:9093
    - kafka.tacocloud.com:9094
```

These configurations are for Kafka bootstrap servers on a host named kafka.tacocloud.com. If you're running your Kafka cluster locally (which is likely during development), then you'll want to use localhost instead, shown next:

```
spring:
  kafka:
    bootstrap-servers:
    - localhost:9092
```

With Kafka set up in your project, you're ready to send and receive messages. You'll start by sending `TacoOrder` objects to Kafka using `KafkaTemplate`.

3.2 Sending messages with KafkaTemplate

In many ways, `KafkaTemplate` is similar to its JMS and RabbitMQ counterparts. At the same time, however, it's very different. This becomes apparent as we consider its methods for sending messages, as shown here:

```
ListenableFuture<SendResult<K, V>> send(String topic, V data);
ListenableFuture<SendResult<K, V>> send(String topic, K key, V data);
ListenableFuture<SendResult<K, V>> send(String topic,
                                        Integer partition, K key, V data);
ListenableFuture<SendResult<K, V>> send(String topic,
                                        Integer partition, Long timestamp, K key, V data);
ListenableFuture<SendResult<K, V>> send(ProducerRecord<K, V> record);
ListenableFuture<SendResult<K, V>> send(Message<?> message);

ListenableFuture<SendResult<K, V>> sendDefault(V data);
ListenableFuture<SendResult<K, V>> sendDefault(K key, V data);
ListenableFuture<SendResult<K, V>> sendDefault(Integer partition,
                                               K key, V data);
ListenableFuture<SendResult<K, V>> sendDefault(Integer partition,
                                               Long timestamp, K key, V data);
```

The first thing you may have noticed is that there are no `convertAndSend()` methods. That's because `KafkaTemplate` is typed with generics and is able to deal with domain types directly when sending messages. In a way, all of the `send()` methods are doing the job of `convertAndSend()`.

You may also have noticed that there are several parameters to `send()` and `sendDefault()` that are quite different from what you used with JMS and Rabbit. When sending messages in Kafka, you can specify the following parameters to guide how the message is sent as follows:

- The topic to which to send the message (required for `send()`)
- A partition to which to write the topic (optional)
- A key to send on the record (optional)
- A timestamp (optional; defaults to `System.currentTimeMillis()`)
- The payload (required)

The topic and payload are the two most important parameters. Partitions and keys have little effect on how you use `KafkaTemplate`, aside from being extra information provided as parameters to `send()` and `sendDefault()`. For our purposes, we're going to fo-

cus on sending the message payload to a given topic and not worry ourselves with partitions and keys.

For the `send()` method, you can also choose to send a `ProducerRecord`, which is little more than a type that captures all of the preceding parameters in a single object. You can also send a `Message` object, but doing so would require you to convert your domain objects into a `Message`. Generally, it's easier to use one of the other methods rather than to create and send a `ProducerRecord` or `Message` object.

Using the `KafkaTemplate` and its `send()` method, you can write a Kafka-based implementation of `OrderMessagingService`. The following listing shows what such an implementation might look like.

Listing 9.8 Sending orders with `KafkaTemplate`

```
package tacos.messaging;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Service;
import tacos.TacoOrder;
@Service
public class KafkaOrderMessagingService
                        implements OrderMessagingService {

  private KafkaTemplate<String, TacoOrder> kafkaTemplate;

  @Autowired
  public KafkaOrderMessagingService(
        KafkaTemplate<String, TacoOrder> kafkaTemplate) {
    this.kafkaTemplate = kafkaTemplate;
  }

  @Override
  public void sendOrder(TacoOrder order) {
    kafkaTemplate.send("tacocloud.orders.topic", order);
  }

}
```

In this new implementation of `OrderMessagingService`, the `sendOrder()` method uses the `send()` method of the injected `KafkaTemplate` to send a `TacoOrder` to the topic named `tacocloud.orders.topic`. Except for the word "Kafka" scattered throughout the

code, this isn't much different than the code you wrote for JMS and Rabbit. And, just like those other implementations of `OrderMessagingService`, it can be injected into `OrderApiController` and used to send orders through Kafka when orders are placed via the /api/orders endpoint.

Until we create a Kafka implementation of the message receiver, you'll need a console to view what was sent. There are several management consoles available for Kafka, including Offset Explorer (**https://www.kafkatool.com/**) and Confluent's Apache Kafka UI (**http://mng.bz/g1P8**).

If you set a default topic, you can simplify the `sendOrder()` method slightly. First, set your default topic to `tacocloud.orders.topic` by setting the `spring.kafka.template.default-topic` property as follows:

```
spring:
  kafka:
    bootstrap-servers:
    - localhost:9092
    template:
      default-topic: tacocloud.orders.topic
```

Then, in the `sendOrder()` method, you can call `sendDefault()` instead of `send()` and not specify the topic name, as shown here:

```
@Override
public void sendOrder(TacoOrder order) {
  kafkaTemplate.sendDefault(order);
}
```

Now that your message-sending code has been written, let's turn our attention to writing code that will receive those messages from Kafka.

3.3 Writing Kafka listeners

Aside from the unique method signatures for `send()` and `sendDefault()`, `KafkaTemplate` differs from `JmsTemplate` and `RabbitTemplate` in that it doesn't offer any methods for receiving messages. That means the only way to consume messages from a Kafka topic using Spring is to write a message listener.

For Kafka, message listeners are defined as methods that are annotated with `@KafkaListener`. The `@KafkaListener` annotation is roughly analogous to `@JmsListener` and `@RabbitListener` and is used in much the same way. The next listing shows what your listener-based order receiver might look like if written for Kafka.

Listing 9.9 Receiving orders with `@KafkaListener`

```
package tacos.kitchen.messaging.kafka.listener;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Component;
import tacos.Order;
import tacos.kitchen.KitchenUI;

@Component
public class OrderListener {

  private KitchenUI ui;

  @Autowired
  public OrderListener(KitchenUI ui) {
    this.ui = ui;
  }

  @KafkaListener(topics="tacocloud.orders.topic")
  public void handle(TacoOrder order) {
    ui.displayOrder(order);
  }

}
```

The `handle()` method is annotated with `@KafkaListener` to indicate that it should be invoked when a message arrives in the topic named `tacocloud.orders.topic`. As it's written in listing 9.9, only a `TacoOrder` (the payload) is given to `handle()`. But if you need additional metadata from the message, it can also accept a `ConsumerRecord` or `Message` object.

For example, the following implementation of `handle()` accepts a `ConsumerRecord` so that you can log the partition and timestamp of the message:

```
@KafkaListener(topics="tacocloud.orders.topic")
public void handle(
```

```
      TacoOrder order, ConsumerRecord<String, TacoOrder> record) {
  log.info("Received from partition {} with timestamp {}",
      record.partition(), record.timestamp());

  ui.displayOrder(order);
}
```

Similarly, you could ask for a `Message` instead of a `ConsumerRecord` and achieve the same thing, as shown here:

```
@KafkaListener(topics="tacocloud.orders.topic")
public void handle(Order order, Message<Order> message) {
  MessageHeaders headers = message.getHeaders();
  log.info("Received from partition {} with timestamp {}",
      headers.get(KafkaHeaders.RECEIVED_PARTITION_ID),
      headers.get(KafkaHeaders.RECEIVED_TIMESTAMP));
  ui.displayOrder(order);
}
```

It's worth noting that the message payload is also available via `ConsumerRecord.value()` or `Message.getPayload()`. This means that you could ask for the `TacoOrder` through those objects instead of asking for it directly as a parameter to `handle()`.

Summary

- Asynchronous messaging provides a layer of indirection between communicating applications, which allows for looser coupling and greater scalability.
- Spring supports asynchronous messaging with JMS, RabbitMQ, or Apache Kafka.
- Applications can use template-based clients (`JmsTemplate`, `RabbitTemplate`, or `KafkaTemplate`) to send messages via a message broker.
- Receiving applications can consume messages in a pull-based model using the same template-based clients.
- Messages can also be pushed to consumers by applying message listener annotations (`@JmsListener`, `@RabbitListener`, or `@KafkaListener`) to bean methods.

---

[1] These methods are defined by `AmqpTemplate`, an interface implemented by `RabbitTemplate`.

Previous chapter

< 8 Securing REST

Next chapter

10 Integrating Spring >